



UNIVERSITATEA DIN
BUCUREȘTI

FACULTATEA DE
MATEMATICĂ ȘI
INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

PLATFORMĂ BAZATĂ PE DOVEZI
PENTRU MONITORIZAREA
ANTRENAMENTELOR,
DESCOPERIREA DE PROGRAME,
COACHING ȘI CĂUTARE ÎN
ISTORICUL DE ANTRENAMENT

Absolvent

[Prenume NUME]

Coordonator științific

[Titlul și numele coordonatorului]

București, iulie 2026

Rezumat

Lucrarea prezintă proiectarea și implementarea unei platforme full-stack pentru antrenamente de forță, construită pe principiul *adevărului istoric*: planurile de antrenament se transformă, la execuție, în istoric imuabil prin *instantanee* (snapshots), astfel încât editarea sau ștergerea ulterioară a datelor sursă nu poate modifica ce a fost înregistrat. Sistemul oferă gestiunea programelor de antrenament, execuția sesiunilor live, analize de progres, un marketplace de programe (publicare, vot, salvare, copiere), o analiză structurală deterministă și explicabilă a programelor (bazată pe euristici informate de dovezi, fără pretenții medicale), un mod de *coaching* cu acces controlat la datele clientului și o componentă de *căutare* full-text peste programe și istoricul de antrenament. Componenta de căutare este un model de citire derivat, construit peste OpenSearch, menținut consistent prin evenimente după-commit, cu securitate în două straturi. Contribuția centrală este o *evaluare empirică* OpenSearch vs. PostgreSQL pe un corpus sintetic scalat până la 2 milioane de documente. Backend-ul este Spring Boot 3 / Java 21 cu PostgreSQL 16 ca sursă unică de adevăr, iar corectitudinea este verificată prin teste de integrare pe baze de date reale (Testcontainers, fără mock-uri). Rezultatele arată un punct de echilibru în jurul a 10.000 de documente și un avantaj decisiv al motorului de căutare la căutarea tolerantă la greșeli de tastare.

Abstract

This thesis presents the design and implementation of a full-stack strength-training platform built on the principle of *historical truth*: when a workout is executed, the plan is frozen into immutable *snapshots*, so later edits or deletions of the source data can never alter what was recorded. The system provides workout-program management, live session execution, progress analytics, a program marketplace (publish, vote, save, copy), a deterministic and explainable structural analysis of programs (evidence-informed heuristics, no medical claims), a *coaching* mode with controlled access to a client's data, and a full-text *search* component over programs and training history. The search component is a derived read model built on OpenSearch, kept consistent through after-commit events, with two-layer security. The central contribution is an *empirical evaluation* of OpenSearch versus PostgreSQL on a synthetic corpus scaled up to two million documents. The backend is Spring Boot 3 / Java 21 with PostgreSQL 16 as the single source of truth, and correctness is verified through integration tests on real databases (Testcontainers, no mocks). The results show a crossover point around ten thousand documents and a decisive advantage for the search engine on typo-tolerant (fuzzy) queries.

Cuprins

1	Introducere	5
1.1	Tipul lucrării și încadrarea temei	5
1.2	Prezentarea generală a temei	5
1.3	Scopul și motivația alegerii temei	6
1.4	Contribuția proprie (pe scurt)	6
1.5	Structura lucrării	7
2	Preliminarii	8
2.1	Tehnologii de bază	8
2.2	Regăsirea informației și căutarea full-text	8
2.3	Modele de citire derivate și consistență eventuală	9
2.4	Imuabilitatea istoricului	9
2.5	Fundamentarea analizorului de programe	10
2.6	Obiectivele lucrării în raport cu contextul	10
3	Contribuția proprie	11
3.1	Cerințe și prezentare generală a sistemului	11
3.2	Arhitectura sistemului	11
3.3	Modelul de date și principiul instantaneelor	12
3.4	Modelul de securitate	13
3.5	Funcționalitățile aplicației	14
3.6	Analizorul structural informat de dovezi	15
3.7	Modelul de citire pentru căutare (OpenSearch)	15
3.8	Evaluarea empirică: OpenSearch vs. PostgreSQL	17
3.9	Modul de coaching: RBAC plus ReBAC	18
3.10	Strategia de testare	18
3.11	Direcții de dezvoltare proiectate	18
4	Concluzii	20
4.1	Concluzii privind realizarea temei	20
4.2	Aprecieri personale privind relevanța rezultatelor	20

4.3 Dezvoltări ulterioare	21
Bibliografie	22
Anexa 1 — Capturi de ecran ale interfeței	24
Anexa 2 — Schema bazei de date	27
Anexa 3 — Listări de cod reprezentative	28
Anexa 4 — Rezultatele complete ale evaluării	30
Anexa 5 — Contractul API (selecție)	32
Anexa 6 — Configurație și rulare	33

Capitolul 1

Introducere

1.1 Tipul lucrării și încadrarea temei

Lucrarea de față este o *lucrare de licență* cu caracter aplicativ, încadrată în subdomeniul ingineriei sistemelor software de tip web (*full-stack*), cu accent pe persistența datelor, modele de citire derivate și regăsirea informației (*information retrieval*). Tema o reprezintă proiectarea și implementarea unei platforme pentru gestiunea antrenamentelor de forță, a cărei componentă centrală, din perspectivă științifică, este un *model de citire pentru căutare* construit peste motorul OpenSearch, însoțit de o evaluare empirică comparativă față de capacitățile de căutare native ale bazei de date relaționale PostgreSQL.

1.2 Prezentarea generală a temei

Aplicațiile uzuale de monitorizare a antrenamentelor se împart, de regulă, în două categorii: jurnale pasive, care doar stochează ce a făcut utilizatorul, și aplicații care oferă recomandări de tip „antrenor cu inteligență artificială”, opace și neverificabile. Prima categorie nu ajută la planificare; a doua sugerează o autoritate pe care nu o poate justifica. În plus, ambele tratează de obicei istoricul ca pe niște date „vii”, dependente de planurile curente: dacă utilizatorul își modifică sau șterge un program, sesiunile trecute pot deveni incoerente.

Platforma propusă adoptă o poziție diferită. În primul rând, tratează *adevărul istoric* ca o cerință de integritate de prim rang: la execuția unui antrenament, planul este „înghețat” în instantanee imuabile, astfel încât istoricul rămâne corect indiferent de schimbările ulterioare. În al doilea rând, oferă *feedback structural explicabil*: o analiză deterministă, bazată pe reguli informate de dovezi din literatura antrenamentului de forță, care nu pretinde a fi sfat medical. În al treilea rând, scalează regăsirea informației dincolo de filtrarea SQL simplă, printr-un model de citire dedicat căutării, și *măsoară* riguros când această alegere de arhitectură se justifică.

1.3 Scopul și motivația alegerii temei

Scopul lucrării este construirea unui sistem de calitate apropiată de cea de producție — nu un prototip de tip „temă de an” — care să demonstreze stăpânirea unor concepte importante din ingineria software: securitate (autentificare cu token-uri, control al accesului bazat pe roluri și pe relații), integritatea datelor (instantanee imuabile, constrângeri la nivel de bază de date), modele de citire derivate și consistență eventuală, precum și o disciplină de testare riguroasă.

Motivația alegerii componentei de căutare ca temă centrală este dublă. Pe de o parte, căutarea full-text tolerantă la greșeli, cu relevanță, fațete și evidențiere a potrivirilor, este o cerință reală pentru un istoric de antrenament de mari dimensiuni („găsește acel antrenament din 2020”). Pe de altă parte, alegerea între a folosi capacitățile native ale bazei de date și a introduce un motor de căutare dedicat este o decizie de arhitectură frecventă și adesea luată „din reflex”. Lucrarea o transformă într-o întrebare măsurabilă: *de la ce scară devine justificat un motor de căutare dedicat?*

1.4 Contribuția proprie (pe scurt)

Contribuția proprie constă în proiectarea și implementarea integrală a sistemului și, în mod particular:

- un model de date relațional cu integritate puternică (instantanee imuabile, ștergere logică, indecși unici parțiali, constrângeri `CHECK`), validat de Hibernate în mod `validate`;
- un model de securitate cu token-uri JWT și rotație a token-urilor de reîmprospătare, control al accesului bazat pe roluri (RBAC) și pe relații (ReBAC) pentru modul de coaching, sigur față de vulnerabilitatea IDOR (răspuns 404, nu 403);
- un analizor structural determinist și explicabil al programelor de antrenament, cu un scor pe 100 de puncte descompus în cinci sub-scoruri și avertismente justificate;
- un model de citire derivat pentru căutare, construit peste OpenSearch, menținut consistent prin evenimente după-commit, cu securitate în două straturi;
- o evaluare empirică OpenSearch vs. PostgreSQL pe un corpus sintetic scalat de la 1.000 la 2.000.000 de documente, cu o linie de bază PostgreSQL corectă (`tsvector+GIN, pg_trgm`);
- o suită de teste de integrare pe baze de date reale (PostgreSQL și OpenSearch prin Testcontainers, fără mock-uri și fără H2).

1.5 Structura lucrării

Lucrarea este organizată după cum urmează. Capitolul 2 (*Preliminarii*) prezintă noțiunile și tehnologiile pe care se bazează tema și stadiul actual al subdomeniului. Capitolul 3 (*Contribuția proprie*) descrie arhitectura sistemului, modelul de date și principiul instantaneelor, modelul de securitate, analizorul informat de dovezi, modelul de citire pentru căutare, evaluarea empirică OpenSearch vs. PostgreSQL, modul de coaching, strategia de testare și direcțiile de dezvoltare deja proiectate și pregătite în schemă. Capitolul 4 (*Concluzii*) sintetizează rezultatele și discută dezvoltările ulterioare. Detaliile voluminoase — capturi de ecran ale interfeței, schema completă a bazei de date, listări de cod, tabelele complete ale evaluării și contractul API — sunt grupate în anexe și referite din text.

Capitolul 2

Preliminarii

Acest capitol prezintă noțiunile și tehnologiile pe care se sprijină lucrarea, stadiul actual al subdomeniului și obiectivele formulate în raport cu acest context.

2.1 Tehnologii de bază

Backend. Aplicația server este construită în Java 21 cu Spring Boot 3 [14], folosind module pentru web (REST), securitate, validare și acces la date prin Spring Data JPA / Hibernate. Persistența se face în PostgreSQL 16 [11], iar schema este gestionată exclusiv prin migrări Flyway. O disciplină importantă este modul `ddl-auto: validate`: Hibernate nu generează și nu modifică niciodată schema, ci doar o validează la pornire, ceea ce surprinde din start orice nepotrivire între entități și schema reală.

Frontend. Interfața este o aplicație de tip *single-page* scrisă în React cu TypeScript [6], livrată chiar de către backend de la aceeași origine, ceea ce simplifică modelul de securitate al cookie-urilor.

Autentificare. Sistemul folosește *JSON Web Tokens* (JWT) [4]: un token de acces de scurtă durată, semnat simetric, transportat în antetul `Authorization`, și un token de reîmprospătare opac, stocat doar ca amprentă criptografică. Parolele sunt stocate ca amprente generate cu o funcție de derivare a cheilor rezistentă la atacuri de tip dicționar.

2.2 Regăsirea informației și căutarea full-text

Regăsirea informației (information retrieval) studiază modul în care se identifică, dintr-o colecție de documente, pe cele relevante pentru o interogare. Spre deosebire de o potrivire exactă SQL, căutarea full-text presupune: analiză lexicală (tokenizare, normalizare, eliminarea diacriticelor), un *index inversat* care leagă termenii de documentele care îi

conțin, o funcție de *relevanță* care ordonează rezultatele și mecanisme de toleranță la erori.

Relevanță și ordonare. Motoarele moderne folosesc funcția de scor *BM25* [9], o rafinare probabilistică a familiei TF-IDF care echilibrează frecvența termenilor cu lungimea documentului. Calitatea ordonării se evaluează cu metrici precum *precision@k* și *nDCG* (normalized discounted cumulative gain) [3], care recompensează plasarea documentelor relevante cât mai sus în lista de rezultate.

Toleranța la greșeli. Căutarea „fuzzy” acceptă potriviri aproximative pe baza distanței de editare (Levenshtein), esențială pentru greșeli de tastare (de exemplu *benhc* → *bench*). În bazele relaționale, un mecanism echivalent este oferit de extensia *pg_trgm* (similaritate pe trigrame).

Căutarea în PostgreSQL vs. motoare dedicate. PostgreSQL oferă căutare full-text nativă prin tipul *tsvector*, funcții de ordonare precum *ts_rank_cd* și indecși GIN [12]. Aceasta este suficientă la scară mică și are avantajul de a păstra totul într-un singur sistem. Motoarele dedicate, precum OpenSearch [7] (un fork open-source al Elasticsearch, construit peste biblioteca Lucene), oferă în schimb analizoare configurabile, sinonime, scoruri BM25, fațete prin agregări și toleranță la greșeli performantă la scară mare. Alegerea între cele două este o decizie de arhitectură pe care lucrarea o tratează experimental.

2.3 Modele de citire derivate și consistență eventuală

Pentru a separa calea de scriere (tranzacțională, normalizată) de calea de citire (optimizată pentru interogări), o practică des întâlnită este construirea unui *model de citire derivat* dintr-o sursă de adevăr unică — o formă relaxată a tiparului CQRS (Command Query Responsibility Segregation) [5]. Modelul derivat (aici, indexul OpenSearch) este *eventual consistent* cu sursa de adevăr (PostgreSQL) și poate fi reconstruit oricând. Problema centrală a acestei abordări este propagarea fiabilă a modificărilor către modelul derivat (problema *dual-write*), discutată în Capitolul 3.

2.4 Imuabilitatea istoricului

Ideea că datele istorice trebuie tratate ca fapte imuabile, niciodată suprascrise, este un principiu de proiectare bine fundamentat în sistemele orientate pe date [2]. În domeniul de față, un antrenament efectuat este un fapt: el nu trebuie să se modifice atunci când planul din care a pornit este editat ulterior. Lucrarea implementează acest principiu prin *instantanee* (snapshots) și chei externe anulabile, detaliate în Secțiunea 3.3.

2.5 Fundamentarea analizorului de programe

Analiza structurală a programelor pornește de la rezultate din literatura antrenamentului de forță, în special relația doză-răspuns dintre volumul săptămânal (numărul de serii pe grupă musculară) și hipertrofie [10]. Aceste rezultate sunt folosite ca *euristici* pentru a semnaliza dezechilibre structurale (de exemplu, volum foarte scăzut sau excesiv, lipsa unui tipar de mișcare), nu ca prescripții individualizate. Analizorul este, prin urmare, determinist, explicabil și prudent, însoțindu-și mereu rezultatul de un avertisment privind limitele sale.

2.6 Obiectivele lucrării în raport cu contextul

Pornind de la acest context, obiectivele lucrării sunt:

1. construirea unei platforme full-stack funcționale, cu integritate puternică a datelor și un model de securitate riguros;
2. proiectarea unui model de citire pentru căutare, derivat și eventual consistent, cu securitate care nu poate fi ocolită din parametrii cererii;
3. evaluarea empirică, pe un corpus scalabil, a momentului în care un motor de căutare dedicat devine justificat față de capacitățile native ale PostgreSQL;
4. demonstrarea corectitudinii prin teste de integrare pe infrastructură reală, nu pe substitute.

Capitolul 3

Contribuția proprie

3.1 Cerințe și prezentare generală a sistemului

Din punct de vedere funcțional, sistemul permite unui utilizator autentificat să își gestioneze datele de planificare (un catalog de exerciții oficiale partajat plus exerciții proprii, rutine de încălzire/încheiere, săli și echipamente), să construiască *programe* de antrenament pe mai multe zile, să execute sesiuni live (înregistrând serii cu greutate, repetări, RPE), să își consulte istoricul și progresul, să publice și să copieze programe într-un *marketplace*, să obțină o analiză structurală a unui program, iar — în calitate de antrenor — să consulte (doar citire) datele unui client cu care are o relație activă. Peste programe și peste istoricul de antrenament funcționează o componentă de *căutare* full-text.

Cerințele nefuncționale au ghidat întreaga implementare: PostgreSQL este sursa unică de adevăr; schema este fixată din start și evoluează doar prin migrări Flyway; fiecare resursă deținută de un utilizator impune verificări de proprietate pentru a evita vulnerabilitatea IDOR; codul țintește o calitate de tip „portofoliu / sisteme back-end”, fără substitute în memorie, fără H2, cu teste de integrare reale. Capturi de ecran ale interfeței sunt prezentate în **Anexa 1**.

3.2 Arhitectura sistemului

Sistemul este o aplicație web pe trei niveluri: o aplicație React (single-page) comunică prin JSON cu un API REST Spring Boot, care persistă în PostgreSQL. În producție, frontend-ul compilat este inclus în backend și servit de la aceeași origine (<http://localhost:8080/>), ceea ce menține cookie-ul de autentificare same-origin. Întregul stack rulează prin Docker Compose (**Anexa 6**).

Backend-ul este organizat *pe funcționalități* (package-by-feature), nu pe niveluri tehnice. Fiecare pachet de funcționalitate (`auth`, `exercise`, `routine`, `gym`, `template`, `session`,

history, analytics, marketplace, analyzer, search, coaching, shared) conține propriile patru niveluri: **domain** (entități JPA), **application** (servicii, comenzi, excepții specifice), **infrastructure** (depozite Spring Data și SQL nativ) și **web** (controllere și DTO-uri). O regulă strictă este că nivelul web nu expune niciodată entități: controllerele schimbă doar DTO-uri. Atunci când o funcționalitate are nevoie de datele alteia (de exemplu analizorul citește structura unui program), producătorul expune un *model de citire la nivel de aplicație*, nu un DTO web sau un depozit, păstrând sănătoasă direcția dependențelor.

O cerere este autentificată de un filtru de tip servlet care validează JWT-ul și populează contextul de securitate cu un **UserPrincipal**; controllerele citesc acest principal și nu au încredere niciodată într-un identificador de utilizator venit din corpul cererii. Erorile de domeniu extind o excepție comună care poartă un cod HTTP și un cod de eroare stabil, citibil de mașină, iar un singur *handler* le transformă într-un plic de eroare uniform (**Anexa 5**). Diagrama de arhitectură este prezentată în Figura 3.1.

*[Substituent imagine] Diagrama de arhitectură: SPA React → API
REST Spring Boot → PostgreSQL (sursa de adevăr) și OpenSearch
(model de citire derivat).
(se va înlocui cu \includegraphics după adăugarea capturii în images/)*

Figura 3.1: Arhitectura de ansamblu a sistemului.

3.3 Modelul de date și principiul instantaneelor

Schema este creată de o singură migrare inițială (V1), care acoperă utilizatorii, catalogul de exerciții și grupele musculare, rutinele, sălile și echipamentele, programele cu copii lor (zile / exerciții / rutine), sesiunile live cu instantaneele lor, tabelele de marketplace (voturi, salvări, evenimente de utilizare, statistici per program) și tabele rezervate unor extinderi ulterioare (coaching, rezultate de analiză, un *outbox*). O a doua migrare (V2) populează catalogul oficial de exerciții, iar a treia (V3) adaugă un index de concurență. Inventarul complet al tabelor și constrângerilor este prezentat în **Anexa 2**.

Principiul instantaneelor. Adevărul istoric este cerința de integritate centrală și este impus pe două niveluri. Întâi, exercițiile dintr-o zi de program memorează **exercise_name_snapshot**, **exercise_type_snapshot** și un instantaneu al grupelor musculare, păstrând doar o referință *anulabilă* către exercițiul viu (ON DELETE SET NULL). Apoi, la

pornirea unui antrenament, se face un al doilea instantaneu: numele programului, al zilei și al sălii, precum și numele/tipul/câmpurile planificate ale fiecărui exercițiu și grupele musculare sunt copiate în `session_exercises`; seriile înregistrate memorează numele echipamentului. Pentru că rândurile sesiunii poartă propriile valori și doar chei externe anulabile, editarea sau ștergerea programului, exercițiului, rutinei, sălii sau echipamentului sursă nu pot modifica niciodată ce a înregistrat o sesiune trecută. Acest comportament este verificat direct prin teste de integrare.

Ștergere logică, unicitate și constrângeri. Resursele de planificare folosesc o coloană `deleted_at`, astfel încât istoricul care le referă supraviețuiește. Unicitatea este impusă cu *indecși unici parțiali* care ignoră rândurile șterse logic (de exemplu, un nume unic de exercițiu propriu per utilizator `WHERE deleted_at IS NULL`), permițând reutilizarea unui nume după ștergere. Numeroase invariante sunt impuse de baza de date prin constrângeri `CHECK`: domenii de valori pentru enumerări, `days_per_week BETWEEN 1 AND 7`, `rpe BETWEEN 1.0 AND 10.0`, `finished_at >= started_at`, contoare nenegative, anti-autocoaching (`coach_user_id <> client_user_id`) și invariantul de vizibilitate publicată a programelor.

Concurență. Regula „un singur antrenament activ per utilizator” este garantată de un *index unic parțial* `ux_workout_sessions_one_active_per_user ... WHERE status = 'IN_PROGRESS'` (migrarea V3); verificarea din serviciu este doar o cale rapidă prietenoasă, iar o cursă pierdută între două cereri concurente apare ca un `409 ACTIVE_WORKOUT_EXISTS` curat, nu ca o eroare 500. Contoarele de marketplace (voturi, salvări) sunt actualizate cu SQL atomic, mărginit (`GREATEST(0, contor +/- delta)`), fără cursă citire–modificare–scriere și fără valori negative.

3.4 Modelul de securitate

Parolele sunt stocate ca amprente BCrypt. La autentificare, API-ul emite un token de acces JWT de scurtă durată (HS256), transmis de SPA în antetul `Authorization` și păstrat doar în memorie, și un token de reîmprospătare opac, stocat ca un cookie `HttpOnly, SameSite=Strict`, limitat la calea `/api/auth`. Token-ul de reîmprospătare este persistat doar ca amprentă SHA-256. La reîmprospătare, token-ul este *rotit* tranzacțional: cel prezentat este validat și revocat, iar o nouă pereche este emisă, astfel încât un token furat și reutilizat este detectat (amprenta sa există, dar are `revoked_at` setat). Protecția CSRF este dezactivată deliberat, deoarece token-ul de acces este purtat în antet, nu în cookie; singurul cookie cu credențiale este cel de reîmprospătare, protejat prin atributele de mai sus.

Autorizare și siguranță IDOR. Fiecare resursă deținută este încărcată cu o interogare care include identificatorul proprietarului. Esențial, o cerere pentru o resursă a altui utilizator întoarce **404**, nu **403**, astfel încât API-ul nu dezvăluie nici măcar existența resursei. Resursele imbricate își rezolvă proprietatea în lanț (de exemplu, un exercițiu dintr-o zi de program este accesibil doar dacă programul zilei aparține apelantului). Ierarhia de autorizare din configurația de securitate este: rute publice de autentificare; `/api/admin/**` doar pentru rol ADMIN; `/api/coach/**` doar pentru rol COACH; restul `/api/**` doar autentificat.

3.5 Funcționalitățile aplicației

Date de planificare și programe. Utilizatorii construiesc o bibliotecă de exerciții (catalogul oficial plus exerciții proprii, fiecare etichetat cu grupe musculare primare/-secundare), rutine, săli și echipamente. Un program este privat și pe mai multe zile; fiecare zi conține exerciții ordonate (cu serii/repetări/greutate/pauză planificate) și rutine atașate. După fiecare modificare a exercițiilor, vectorii agregați ai programului (grupe musculare, identificatori de exerciții oficiale, nume de exerciții) sunt recalculați pe partea de PostgreSQL printr-o singură instrucțiune nativă cu `array_agg(DISTINCT ... ORDER BY ...)`, pregătind căutarea.

Execuția live și istoricul. Pornirea unui antrenament execută, într-o singură tranzacție, copierea-instantaneu din Secțiunea 3.3. În timpul sesiunii, utilizatorul înregistrează serii, adaugă exerciții suplimentare neplanificate și, în final, finalizează sau anulează sesiunea. Istoricul listează sesiunile cele mai recente întâi, cu numere de sumar per sesiune calculate printr-o singură interogare în lot (evitând problema N+1).

Analize de progres. Analizele sunt calculate integral în PostgreSQL peste sesiunile finalizate: volumul total pe zi, numărul de antrenamente pe săptămână (`date_trunc`), distribuția seriilor pe grupa musculară primară și, pentru un grafic de progresie a forței, cea mai bună estimare a unei repetări maxime (1RM) pe exercițiu pe zi, folosind formula lui Epley [1]:

$$1RM \approx w \cdot \left(1 + \frac{r}{30}\right), \quad (3.1)$$

unde w este greutatea și r numărul de repetări ale seriei.

Marketplace. Proprietarii publică un program nevid, făcându-l public. Utilizatorii autentificați parcurg programele publice (sortate după cele mai noi, scor sau tendință), votează, salvează și *folosesc* un program. „Folosirea” copiază în profunzime programul public într-un program nou, privat, deținut de cel care copiază; referința către exercițiu este păstrată doar când este un exercițiu *oficial*, altfel este anulată, dar instantaneele de

nume/tip/grupe musculare sunt păstrate — deci copia este complet utilizabilă și independentă de sursă, fără scurgeri de date private.

3.6 Analizorul structural informat de dovezi

Analizorul este determinist și bazat pe reguli — fără învățare automată. Produce un *Scor de structură a programului* (nu o judecată de „calitate științifică” sau medicală) și fiecare răspuns poartă un avertisment că nu este sfat medical și că ignoră recuperarea, tehnica, efortul, somnul, alimentația și răspunsul individual. Este calculat live la un `GET read-only (/api/templates/{id}/analysis)` și este folosit și la indexarea pentru căutare.

Volumul săptămânal este suma peste zilele autorizate ale programului, iar seriile sunt ponderate după rolul muscular (primar 1.0, secundar 0.5). Exercițiile fără serii planificate sunt *excluse* din volum (niciodată presupuse implicit) și marcate ca date incomplete; doar exercițiile de forță contează la volum și echilibru. Scorul de 100 de puncte este împărțit în cinci sub-scoruri, fiecare pornind de la maximul său și scăzând o valoare fixă pentru fiecare constatare (mărginit la zero), astfel încât rezultatele sunt complet deterministe și testabile (Tabelul 3.1). Categoriile mapează totalul la `WELL_STRUCTURED` (≥ 80), `DECENT_STRUCTURE` (55–79) sau `NEEDS_REVIEW` (< 55). Familiile de reguli și o listare reprezentativă a logicii de scor sunt detaliate în **Anexa 3**.

Sub-scor	Maxim	Exemple de penalizări
Volum și acoperire	35	regiune absentă −12; volum excesiv −5; foarte scăzut −3
Frecvență	20	volum concentrat într-o singură zi −4
Echilibru	20	dezechilibru sever −8; moderat −4
Designul sesiunii	15	sesiune excesivă −5; concentrare pe o grupă −3
Specificitate / pauză	10	pauză prea scurtă −2; interval de repetări extrem −2 ... −3

Tabela 3.1: Descompunerea scorului de structură pe cinci sub-scoruri deterministe.

3.7 Modelul de citire pentru căutare (OpenSearch)

Căutarea este un *model de citire derivat* construit peste OpenSearch, activat printr-un indicator de configurare (`app.search.enabled`, implicit dezactivat, astfel încât compilarea locală și suita de teste PostgreSQL rămân independente de OpenSearch). Există două indexuri, `templates` și `workout_sessions`, cu nume versionate ascunse în spatele unor alias-uri stabile. Analizorul de indexare normalizează textul (standard + lowercase + eliminarea diacriticelor), iar cel de căutare adaugă un graf de *sinonime* (de exemplu *pecs*→*chest*, *ohp*→*overhead press*). Interogarea de tip `multi_match` aplică ponderi (boost) pe câmpuri — pentru programe `name^4`, `exerciseNames^3`, `muscleGroupsText^2`

etc. — și toleranță la greșeli (`fuzziness=AUTO`, cu o gardă care nu aplică „fuzzy” interogărilor mai scurte de 3 caractere). Pentru marketplace, scorul de relevanță este combinat cu un `function_score` care adaugă o influență logaritmică a popularității (salvări, utilizări). Rezultatele includ fațete (agregări `terms` și un histogram pe lună) și evidențierea potrivirilor cu `<mark>`.

Calea de scriere și consistența. Indexarea este declanșată de evenimente după-commit (`@TransactionalEventListener(AFTER_COMMIT)`): astfel, indexul nu reflectă niciodată munca revenită prin rollback. Două căi de actualizare codifică contractul de consistență: o reindexare *structurală* reconstruiește documentul complet și recalculează câmpurile derivate din analiză, în timp ce o actualizare *doar a contoarelor* (vot/salvare/utilizare) nu recalculează niciodată analiza. Toate scrierile sunt *idempotente* (upsert al documentului întreg după identificator). Eșecurile sunt jurnalizate și înghițite — PostgreSQL rămâne sursa de adevăr, iar o reconstrucție le recuperează. Aceasta este, deliberat, *nu* un outbox tranzacțional (vezi Secțiunea 3.11).

Securitate în două straturi. Întâi, un *filtru de domeniu obligatoriu*, derivat din JWT, pe care parametrii cererii nu îl pot suprascrie: `marketplace` $\Rightarrow$ `visibility=PUBLIC`; `my` $\Rightarrow$ `ownerUserId=apelant`; `workouts` $\Rightarrow$ `ownerUserId=apelant`. Apoi, o *apărare în adâncime* pe partea PostgreSQL: identificatorii returnați de OpenSearch sunt revalidați în baza de date cu aceeași regulă, eliminând orice rezultat învechit (de exemplu, recent depublicat sau șters) și fuzionând starea vie de vot/salvare pentru marketplace.

Reconstrucție și reziliență. Serviciul de reconstrucție creează un index nou cu marcaj temporal, îl populează în lot din PostgreSQL și *schimbă atomic alias-ul* (fără timp de nefuncționare la citire), apoi șterge indexul vechi; la eșec, indexul nou este eliminat, iar cel vechi continuă să servească. Declanșarea manuală (`POST /api/admin/search/reindex`) este permisă doar rolului ADMIN. Când căutarea este dezactivată sau OpenSearch este indisponibil, endpoint-urile întorc 503 și interfața revine la lista SQL; un index încă neconstruit este tratat ca „fără rezultate”, nu ca o eroare.

Căutarea de către antrenor. Un antrenor poate căuta full-text în istoricul unui client (`/api/coach/clients/{id}/search/workouts`). Este singurul loc în care căutarea rulează cu un identificator de proprietar diferit de apelant și este sigură pentru că trece mai întâi prin poarta de acces a antrenorului (vezi Secțiunea 3.9) și apoi reutilizează exact același serviciu de căutare cu identificatorul clientului, păstrând filtrul de domeniu și revalidarea în PostgreSQL.

3.8 Evaluarea empirică: OpenSearch vs. PostgreSQL

Contribuția centrală, din perspectivă științifică, este o evaluare empirică a momentului în care un motor de căutare dedicat devine justificat față de capacitățile native ale PostgreSQL.

Metodologie. Linia de bază PostgreSQL este *corectă*, nu un om de paie: căutare full-text ponderată cu `tsvector` (`setweight A/B/C/D`) și index GIN, ordonată cu `ts_rank_cd`; căutare fuzzy cu `pg_trgm` și index GIN; fațete prin `GROUP BY`. Partea OpenSearch folosește exact aceleași mapări și interogări ca producția. Corpusul sintetic este generat determinist și *în flux* (pe loturi, cu memorie constantă), cu un vocabular realist de ridicări, popularitate înclinată și 90%/10% public/privat. Au fost evaluate șapte dimensiuni, de la 1.000 la 2.000.000 de documente, pe cinci scenarii: două căutări full-text, o căutare fuzzy (greșeală de tastare), o căutare filtrată și o agregare pe fațete. Măsurarea este monofir, cu încălzire și un buget de timp, raportând percentilele $p_{50}/p_{95}/p_{99}$.

Rezultate. Tabelul 3.2 prezintă latențele la cea mai mare scară (2M de documente); tabelele complete pe toate dimensiunile se află în **Anexa 4**. Concluziile sunt: există un *punct de echilibru în jurul a 10.000 de documente* (sub el, PostgreSQL este mai rapid; peste el, OpenSearch câștigă decisiv); căutarea full-text în PostgreSQL crește aproximativ liniar (de la $\sim 1,5$ ms la $\sim 1,86$ s între 1k și 2M), în timp ce OpenSearch rămâne plat (~ 3 – 64 ms); *căutarea fuzzy este diferența decisivă* — `pg_trgm` se degradează de la 14 ms la $\sim 15,8$ s (1k $\rightarrow$ 2M), față de ~ 4 ms, practic constant, în OpenSearch (un raport de ordinul a $4000\times$ la 2M); în plus, indexul OpenSearch ocupă de $\sim 3,6\times$ mai puțin spațiu pe disc și se construiește de $\sim 2\times$ mai repede la 2M.

Scenariu (la 2M documente)	PostgreSQL	OpenSearch
Full-text „bench press” (p_{50})	1864 ms	64 ms
Full-text „squat” (p_{50})	1437 ms	4 ms
Fuzzy „benhc” (greșeală) (p_{50})	15825 ms	4 ms
Filtrat (press + INTERMEDIATE + PPL) (p_{50})	2295 ms	7 ms
Fațetă după dificultate (p_{50})	1193 ms	2 ms
Timp de construcție a indexului	239 s	115 s
Dimensiune pe disc	2632 MB	739 MB

Tabela 3.2: Comparație OpenSearch vs. PostgreSQL la 2.000.000 de documente. Tabelele complete pe toate cele șapte dimensiuni sunt în Anexa 4.

Interpretare. Rezultatul transformă o alegere de arhitectură luată adesea „din reflex” într-o decizie informată: sub pragul de echilibru, complexitatea suplimentară a unui motor

dedicat nu se justifică; peste el — și mai ales dacă este necesară toleranța la greșeli — un motor dedicat devine nu doar util, ci necesar.

3.9 Modul de coaching: RBAC plus ReBAC

Modul de coaching adaugă un al doilea strat de control al accesului peste RBAC. O relație antrenor–client are un ciclu de viață (`PENDING` → `ACTIVE` prin acceptare, sau `REJECTED`; o relație `ACTIVE` poate fi `REVOKED` de oricare parte), cu un index unic parțial care permite o singură relație vie per pereche. RBAC (`/api/coach/**` cere rolul `COACH`) este necesar, dar nu suficient: fiecare citire per client trece printr-o poartă de control bazat pe relații (ReBAC) care cere o relație *activă*, eșuând altfel cu **404 (nu 403)**, astfel încât un antrenor nu poate sonda ce utilizatori există. Accesul antrenorului este strict de citire (istoric, analize, o sesiune, căutare); nu există niciun endpoint de antrenor care să modifice datele clientului, iar clientul poate revoca accesul oricând.

3.10 Strategia de testare

Corectitudinea este verificată prin teste de integrare pe *infrastructură reală*: o clasă de bază pornește un container PostgreSQL (Testcontainers), iar suita de căutare adaugă un container OpenSearch real și activează indicatorul de căutare — niciun mock, niciun H2. Astfel, întreaga schemă gestionată de Flyway și constrângerile sale sunt exercitate. Testele afirmă comportamentul prin nivelul HTTP și, unde contează, garanții la nivel de bază de date: o a doua inserare „în desfășurare” brută este respinsă de indexul unic parțial; vectorii agregați sunt citați înapoi cu `JdbcTemplate`; instanțele unei sesiuni rămân neschimbate după editarea datelor sursă. La momentul redactării, suita conține 128 de teste care trec.

3.11 Direcții de dezvoltare proiectate

Următoarele componente sunt *proiectate și pregătite*, dar nu încă implementate. Sunt incluse aici pentru că arhitectura și, în mai multe cazuri, chiar schema bazei de date le anticipează: schema `V1` conține deja, neutilizate de cod, tabelele `outbox_events`, `template_analysis_results`, `coach_session_comments` și `coach_template_assignments`.

- **Cache cu Redis pentru marketplace (cache-aside).** Memorarea paginilor de listă și a detaliilor de program, cu invalidare condusă de aceleași evenimente după-commit existente, și protecție împotriva efectului de „turmă” prin expirare probabilistică timpurie (XFetch [13]).

- **Outbox tranzacțional pentru indexare.** Rezolvarea problemei *dual-write*: un rând de eveniment scris în aceeași tranzacție cu modificarea, în tabela `outbox_events` deja existentă, plus un releu programat cu livrare cel-puțin-o-dată și un consumator idempotent (indexatorul este deja idempotent). Se discută alternativa CDC/Debezium [8].
- **Evaluarea calității relevanței.** Extinderea evaluării de la viteză la calitate: un set de judecăți etichetate și metrice precum `precision@k` și `nDCG` [3], cu un studiu de ablație (sinonime / fuzzy / ponderi pornite/oprite).
- **Paginare prin chei (keyset).** Înlocuirea paginării `OFFSET` actuale cu paginare prin cheie de continuare, reducând costul paginilor adânci de la $\mathcal{O}(n)$ la $\mathcal{O}(\log n)$.
- **Limitarea ratei la autentificare** (token-bucket susținut de Redis) și **migrarea la Argon2id** a stocării parolelor, cu re-hashing transparent la autentificare.
- **Observabilitate** (trasare distribuită OpenTelemetry + metrice Prometheus/Grafana), **partiționarea temporală** a tabelii `workout_sessions`, o **coadă de lucru** bazată pe `SELECT ... FOR UPDATE SKIP LOCKED` (potrivită pentru releul de outbox) și **vederi materializate** pentru analize.

Aceste direcții sunt detaliate, ca proiectare, în Capitolul 4.

Capitolul 4

Concluzii

4.1 Concluzii privind realizarea temei

Lucrarea a realizat o platformă coerentă, centrată pe integritatea datelor, pentru antrenamentele de forță: planurile sunt transformate în istoric imuabil prin instantanee, analizele și feedback-ul sunt derivate onest din date reale, iar un marketplace permite partajarea și copierea independentă a programelor. Ingineria a pus accent pe corectitudine (teste pe baze de date reale), integritatea datelor (instantanee pe două niveluri, ștergere logică, constrângeri la nivel de bază de date), securitate (autentificare JWT cu rotație și acces sigur față de IDOR, RBAC plus ReBAC pentru coaching) și explicabilitate (un analizor determinist, însoțit de avertismente), în locul unor funcționalități superficiale.

Componenta de căutare a fost tratată ca o problemă de inginerie a sistemelor: un model de citire derivat, eventual consistent și reconstruibil, cu securitate care nu poate fi ocolită din parametrii cererii. Cea mai importantă contribuție din punct de vedere științific este *evaluarea empirică* OpenSearch vs. PostgreSQL pe un corpus scalat până la două milioane de documente, care a identificat un punct de echilibru în jurul a 10.000 de documente și a arătat că diferența decisivă apare la căutarea tolerantă la greșeli, unde abordarea relațională se degradează la mai multe secunde, în timp ce motorul dedicat rămâne practic constant. Astfel, o decizie de arhitectură luată adesea „din reflex” a fost transformată într-una informată și măsurabilă.

4.2 Aprecieri personale privind relevanța rezultatelor

Consider că valoarea lucrării nu stă în numărul de funcționalități, ci în disciplina cu care au fost construite: o singură sursă de adevăr validată strict, o suită de teste de integrare pe infrastructură reală și o evaluare cantitativă a unei alegeri de arhitectură. Aceste rezultate sunt direct transferabile: tiparul de instantanee pentru adevăr istoric, modelul de citire derivat și consistent și metodologia de comparare a soluțiilor de căutare se aplică

multor altor domenii (comerț electronic, sisteme de documente, jurnale de activitate).

4.3 Dezvoltări ulterioare

Arhitectura a fost proiectată pentru extindere, iar schema bazei de date anticipează deja mai multe direcții (tabelele `outbox_events`, `template_analysis_results`, `coach_session_comments` și `coach_template_assignments` există din prima migrare, încă neutilizate de cod). Direcțiile imediate, prezentate ca proiectare în Secțiunea 3.11, sunt: (1) un strat de cache Redis de tip cache-aside pentru marketplace, cu invalidare condusă de evenimente și protecție împotriva efectului de turmă; (2) un *outbox tranzacțional* pentru indexare, care rezolvă problema dual-write reutilizând tabela și indexatorul idempotent existente; (3) o evaluare a calității relevanței (precision@k , nDCG), complementară evaluării de viteză deja realizate; (4) paginare prin chei (keyset) pentru pagini adânci; (5) limitarea ratei la autentificare și migrarea la Argon2id; și (6) observabilitate prin trasare distribuită și metrice, partiționarea temporală a istoricului, o coadă de lucru bazată pe `SKIP LOCKED` și vederi materializate pentru analize. Fiecare dintre acestea extinde, nu remodelează, fundația existentă.

Bibliografie

- [1] Boyd Epley, *Poundage Chart. Boyd Epley Workout*, Lincoln, NE: Body Enterprises, 1985.
- [2] Pat Helland, „Immutability Changes Everything”, în *Communications of the ACM* 59.1 (2016), pp. 64–70, DOI: [10.1145/2844112](https://doi.org/10.1145/2844112).
- [3] Kalervo Järvelin și Jaana Kekäläinen, „Cumulated Gain-based Evaluation of IR Techniques”, în *ACM Transactions on Information Systems* 20.4 (2002), pp. 422–446, DOI: [10.1145/582415.582418](https://doi.org/10.1145/582415.582418).
- [4] Michael B. Jones, John Bradley și Nat Sakimura, *JSON Web Token (JWT)*, RFC 7519, Internet Engineering Task Force (IETF), 2015, URL: <https://www.rfc-editor.org/rfc/rfc7519> (accesat în 16.06.2026).
- [5] Martin Kleppmann, *Designing Data-Intensive Applications*, O'Reilly Media, 2017, ISBN: 978-1449373320.
- [6] Meta Open Source, *React Documentation*, 2024, URL: <https://react.dev/> (accesat în 16.06.2026).
- [7] OpenSearch Project, *OpenSearch Documentation*, 2024, URL: <https://opensearch.org/docs/latest/> (accesat în 16.06.2026).
- [8] Chris Richardson, *Microservices Patterns: With Examples in Java*, Manning Publications, 2018, ISBN: 978-1617294549.
- [9] Stephen Robertson și Hugo Zaragoza, „The Probabilistic Relevance Framework: BM25 and Beyond”, în *Foundations and Trends in Information Retrieval* 3.4 (2009), pp. 333–389, DOI: [10.1561/15000000019](https://doi.org/10.1561/15000000019).
- [10] Brad J. Schoenfeld, Dan Ogborn și James W. Krieger, „Dose-response Relationship Between Weekly Resistance Training Volume and Increases in Muscle Mass: A Systematic Review and Meta-analysis”, în *Journal of Sports Sciences* 35.11 (2017), pp. 1073–1082, DOI: [10.1080/02640414.2016.1210197](https://doi.org/10.1080/02640414.2016.1210197).
- [11] The PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, 2024, URL: <https://www.postgresql.org/docs/16/> (accesat în 16.06.2026).

- [12] The PostgreSQL Global Development Group, *PostgreSQL Documentation: Full Text Search*, 2024, URL: <https://www.postgresql.org/docs/16/textsearch.html> (accesat în 16.06.2026).
- [13] Andrea Vattani, Flavio Chierichetti și Keegan Lowenstein, „Optimal Probabilistic Cache Stampede Prevention”, în *Proceedings of the VLDB Endowment* 8.8 (2015), pp. 886–897, DOI: [10.14778/2757807.2757813](https://doi.org/10.14778/2757807.2757813).
- [14] VMware Tanzu, *Spring Boot Reference Documentation*, 2024, URL: <https://docs.spring.io/spring-boot/index.html> (accesat în 16.06.2026).

Anexa 1 — Capturi de ecran ale interfeței

Această anexă conține capturi de ecran ale aplicației web. Substituenții de mai jos se vor înlocui cu imaginile reale (`\includegraphics`) după capturarea lor în `images/`.

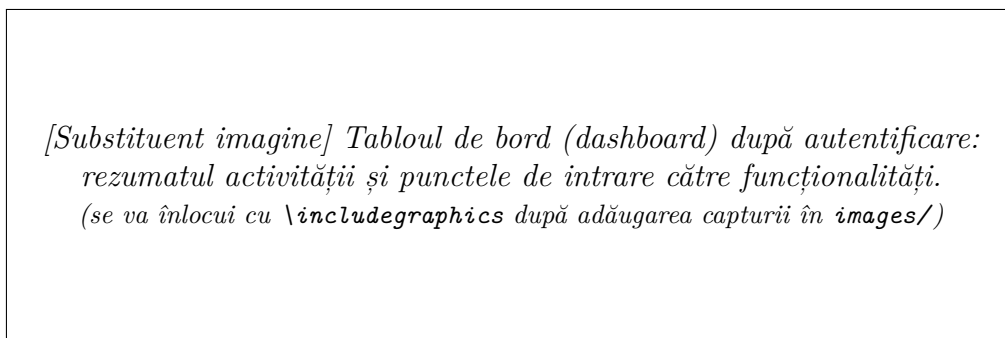


Figura 4.1: Tabloul de bord al utilizatorului.

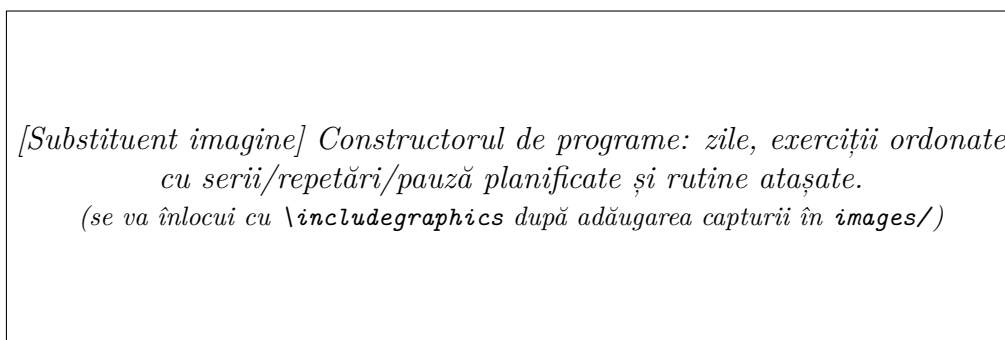


Figura 4.2: Constructorul de programe de antrenament.

*[Substituent imagine] Sesiunea live: înregistrarea seriilor (greutate, repetări, RPE) și adăugarea de exerciții suplimentare.
(se va înlocui cu \includegraphics după adăugarea capturii în images/)*

Figura 4.3: Execuția unei sesiuni de antrenament.

*[Substituent imagine] Analize de progres: volum în timp, antrenamente pe săptămână, distribuția pe grupe musculare și progresia 1RM estimat (grafice Recharts).
(se va înlocui cu \includegraphics după adăugarea capturii în images/)*

Figura 4.4: Pagina de analize și progres.

*[Substituent imagine] Marketplace cu căutare: câmp de interogare, fațete (dificultate, split, grupe musculare) și evidențierea potrivirilor (<mark>).
(se va înlocui cu \includegraphics după adăugarea capturii în images/)*

Figura 4.5: Marketplace-ul de programe cu căutare full-text și fațete.

*[Substituent imagine] Modul de coaching: căutarea full-text în istoricul unui client, accesibilă doar pe baza unei relații active.
(se va înlocui cu \includegraphics după adăugarea capturii în images/)*

Figura 4.6: Căutarea în istoricul clientului (mod coaching).

*[Substituent imagine] Panoul analizorului structural: scorul pe 100 de puncte, sub-scorurile și avertismentele explicabile cu sugestii.
(se va înlocui cu `\includegraphics` după adăugarea capturii în `images/`)*

Figura 4.7: Rezultatul analizorului structural de programe.

Anexa 2 — Schema bazei de date

Schema este definită integral de migrarea Flyway V1 (28 de tabele), populată cu date oficiale de V2 și completată cu un index de concurență de V3. Tabelele, grupate pe funcționalități, sunt:

- **Autentificare / coaching:** `app_users`, `refresh_tokens`, `coach_profiles`, `coach_client_relationships`.
- **Catalog / planificare:** `muscle_groups`, `exercises`, `exercise_muscle_groups`, `routines`, `gyms`, `equipment`.
- **Programe:** `workout_templates`, `template_stats`, `template_days`, `template_day_exercises`, `template_day_exercise_muscle_groups`, `template_day_routines`.
- **Sesiuni (istoric imuabil):** `workout_sessions`, `session_exercises`, `session_exercise_muscle_groups`, `session_routines`, `workout_sets`.
- **Marketplace:** `template_votes`, `template_saves`, `template_use_events`.
- **Tabele pregătite pentru extinderi (neutilizate încă de cod):** `coach_session_comments`, `coach_template_assignments`, `template_analysis_results`, `outbox_events`.

Constrângerile cu rol esențial în integritate sunt: invariantul de vizibilitate publicată a programelor (`CHECK ((visibility='PUBLIC' AND published_at IS NOT NULL) OR visibility='PRIVATE')`); anti-autocoaching (`CHECK (coach_user_id <> client_user_id)`); indexul unic parțial pentru un singur antrenament activ per utilizator; indecșii unici parțiali de unicitate cu `WHERE deleted_at IS NULL`; și cheile externe anulaabile (`ON DELETE SET NULL`) către datele sursă, pereche cu coloanele de instantaneu. Definițiile-cheie sunt în Anexa 3.

Anexa 3 — Listări de cod reprezentative

Listarea 1. Stilul determinist al analizorului: fiecare sub-scor pornește de la maxim și scade o valoare fixă pe constatare (extras simplificat din `TemplateAnalyzerService`).

```
private int scoreVolumeAndCoverage() {
    int score = 35;
    // O grupa minora nu poate masca o regiune majora absenta:
    if (!lowerBody) { add("NO_LOWER_BODY", HIGH, ...); score -= 12; }
    if (!pulling)    { add("NO_PULL",      HIGH, ...); score -= 12; }
    if (!pushing)    { add("NO_PUSH",      HIGH, ...); score -= 12; }
    for (String bucket : ALL_BUCKETS) {
        // serii saptamanale ponderate: PRIMARY = 1.0, SECONDARY = 0.5
        double volume = weeklyWeightedSets(bucket);
        if      (volume <= 3) { add("MUSCLE_VOLUME_VERY_LOW", ...); score -= 3; }
        else if (volume > 25) { add("MUSCLE_VOLUME_EXCESSIVE", ...); score -= 5; }
        else if (volume > 20) { add("MUSCLE_VOLUME_HIGH", ...); score -= 2; }
        // ... alte benzi de volum ...
    }
    return Math.max(0, score);
}

private static String category(int score) {
    if (score >= 80) return "WELL_STRUCTURED";
    return score >= 55 ? "DECENT_STRUCTURE" : "NEEDS_REVIEW";
}
```

Listarea 2. Ierarhia de autorizare (din `SecurityConfig`): RBAC pe rute, peste care modul de coaching adaugă verificarea ReBAC la nivel de serviciu.

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers(POST, "/api/auth/register", "/api/auth/login",
                        "/api/auth/refresh", "/api/auth/logout").permitAll()
    .requestMatchers(GET, "/api/health").permitAll()
    .requestMatchers("/api/admin/**").hasRole("ADMIN")
    .requestMatchers("/api/coach/**").hasRole("COACH")
    .requestMatchers("/api/**").authenticated()
    .anyRequest().permitAll())
```

Listarea 3. Definiții SQL cheie: garda de concurență (V3) și tabela de outbox pregătită din V1 pentru extinderea cu outbox tranzacțional.

```
-- Un singur antrenament activ per utilizator (migrarea V3):
CREATE UNIQUE INDEX ux_workout_sessions_one_active_per_user
  ON workout_sessions (user_id) WHERE status = 'IN_PROGRESS';

-- Tabela de outbox, definita in V1, neutilizata inca de cod:
CREATE TABLE outbox_events (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  aggregate_type varchar(100) NOT NULL,
  aggregate_id uuid NOT NULL,
  event_type   varchar(100) NOT NULL,
  payload      jsonb NOT NULL DEFAULT '{}'::jsonb,
  status       varchar(30) NOT NULL DEFAULT 'PENDING',
  attempts     int NOT NULL DEFAULT 0,
  available_at timestamptz NOT NULL DEFAULT now(),
  processed_at timestamptz,
  created_at   timestamptz NOT NULL DEFAULT now(),
  last_error   text,
  CONSTRAINT check_outbox_status
    CHECK (status IN ('PENDING', 'PROCESSING', 'PROCESSED', 'FAILED'))
);
CREATE INDEX outbox_events_processing_idx
  ON outbox_events (status, available_at, created_at);
```

Anexa 4 — Rezultatele complete ale evaluării

Măsurătorile au fost realizate la 7 dimensiuni ale corpusului (1k–2M), cu 300 de iterații măsurate (încălzire 30, buget de timp 15 s; n indică numărul de eşantioane când bugetul a limitat celula). PostgreSQL: `tsvector`+GIN ordonat cu `ts_rank_cd`, fuzzy cu `pg_trgm`. OpenSearch: aceleași mapări și interogări ca producția.

Corpus	Construcție PG	Construcție OS	Dimensiune PG	Dimensiune OS
1.000	216 ms	203 ms	1,6 MB	0,3 MB
10.000	1.926 ms	975 ms	14,6 MB	2,9 MB
50.000	7.966 ms	3.481 ms	68,7 MB	14,3 MB
100.000	14.006 ms	6.349 ms	135,5 MB	28,5 MB
500.000	68.752 ms	37.987 ms	664,7 MB	140,8 MB
1.000.000	125.774 ms	57.701 ms	1.313,6 MB	349,1 MB
2.000.000	239.192 ms	114.908 ms	2.632,5 MB	738,7 MB

Tabela 4.1: Timpul de construcție a indexului și dimensiunea pe disc, pe dimensiuni de corpus.

Corpus	PostgreSQL ($p_{50}/p_{95}/p_{99}$)	OpenSearch ($p_{50}/p_{95}/p_{99}$)
1.000	1,46 / 2,68 / 3,12	3,25 / 5,70 / 10,56
10.000	10,64 / 14,32 / 17,63	2,85 / 5,21 / 7,17
50.000	48,70 / 66,59 / 77,97	5,17 / 10,05 / 13,98
100.000	100,21 / 117,87 / 136,46	6,84 / 10,59 / 12,83
500.000	779,47 / 1021,25 / 1229,16	22,75 / 38,09 / 67,26
1.000.000	872,19 / 1199,43 / 1220,09	33,64 / 45,24 / 83,90
2.000.000	1863,97 / 2783,12 / 2783,12	63,65 / 73,05 / 111,67

Tabela 4.2: Latența căutării full-text „bench press” (ms), pe dimensiuni de corpus.

Corpus	PostgreSQL ($p_{50}/p_{95}/p_{99}$)	OpenSearch ($p_{50}/p_{95}/p_{99}$)
1.000	14,45 / 18,91 / 21,69	2,37 / 3,74 / 5,63
10.000	127,93 / 134,06 / 136,22	3,23 / 7,38 / 10,94
50.000	621,74 / 645,63 / 646,08	2,66 / 4,95 / 6,59
100.000	1831,47 / 2169,61 / 2169,61	3,15 / 5,74 / 7,38
500.000	2447,22 / 8354,43 / 8354,43	4,06 / 7,20 / 8,95
1.000.000	7806,25 / 17790,10 / 17790,10	3,94 / 9,18 / 13,35
2.000.000	15825,49 / 16399,71 / 16399,71	3,89 / 9,38 / 14,14

Tabela 4.3: Latența căutării fuzzy „benhc” (greșeală de tastare, ms): degradarea `pg_trgm` vs. latența practic constantă a OpenSearch.

Anexa 5 — Contractul API (selecție)

Toate endpoint-urile, cu excepția celor de autentificare și a stării de sănătate, cer un token JWT valid. Răspunsurile de eroare folosesc un plic uniform cu un cod stabil (de exemplu TEMPLATE_NOT_FOUND, ACTIVE_WORKOUT_EXISTS, SEARCH_UNAVAILABLE).

Autentificare	: POST /api/auth/{register,login,refresh,logout}; GET /api/auth/me
Planificare	: /api/exercises; /api/routines; /api/gyms; /api/gyms/{id}/equipment; /api/equipment/{id}
Programe	: /api/templates; /api/templates/{id}; POST /api/templates/{id}/{publish,unpublish}
Sesiuni	: POST /api/workouts/start; GET /api/workouts/active; POST /api/workouts/{id}/{finish,cancel}; POST /api/session-exercises/{id}/sets; PUT DELETE /api/sets/{id}
Istoric	: GET /api/history; GET /api/analytics/overview
Marketplace	: GET /api/marketplace/templates; POST /api/marketplace/templates/{id}/{vote,save,use}
Analizor	: GET /api/templates/{id}/analysis
Cautare	: GET /api/search/templates; GET /api/search/workouts; POST /api/admin/search/reindex (doar ADMIN)
Coaching	: /api/coach/clients/... (rol COACH + relatie activa); /api/coaching/{invites,coaches}

Anexa 6 — Configurație și rulare

Stack-ul rulează prin Docker Compose (PostgreSQL 16, OpenSearch 2.13 cu un volum persistent, și aplicația). Indicatorul `app.search.enabled` controlează activarea căutării.

```
# application.yml (extras)
spring:
  jpa:
    hibernate:
      ddl-auto: validate          # schema detinuta de Flyway
app:
  security:
    jwt:
      issuer: workout-thesis
      access-ttl: 15m
      refresh-ttl: 14d
  search:
    enabled: ${APP_SEARCH_ENABLED:false}
    uri: ${APP_SEARCH_URI:http://localhost:9200}
    templates-index: templates_v1
    sessions-index:  workout_sessions_v1
```

Rularea cu date demonstrative (istoric 2020–prezent) se face cu profilul `demo`, care la pornire reconstruiește indexul de căutare:

```
SPRING_PROFILES_ACTIVE=demo docker compose up --build
```